



Cofinanciado por
la Unión Europea



MINISTERIO
DE HACIENDA



GodCloud

Documento Oficial

Autor:

Francisco Javier
Barea Fernández

2º ASIR

CURSO: 2025/26



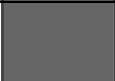
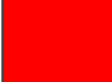
Indice

1. Introducción a GodCloud.....	4
1.01. ¿Cómo empezó todo?.....	4
1.02. Repositorio y Gestión del Proyecto.....	4
2. Objetivos del Proyecto.....	5
2.01. Objetivo General.....	5
2.02. Objetivos Específicos.....	5
3. Diseño de la Estructura de red.....	7
3.01. Diseño Lógico de la Red.....	7
3.02. Servicios de redundancia en capa 2.....	12
3.03. Servicios de redundancia en capa 3.....	13
3.04. Servicios de redundancia en capa 7.....	14
4. Configuración de los Switches Cisco.....	15
4.01. Configuración de las VLANs.....	15
4.02. Configuración de las interfaces en cada Switch.....	16
5. Configuración de los Routers Pfsense.....	22
5.01. Interfaz de Sincronización CARP – em1.....	23
5.02. Agrupación de Interfaces – LAGG Failover.....	23
5.03. Segmentación Virtual – VLANs sobre LAGG.....	24
5.04. Asignación y Direccionamiento de Interfaces.....	24
5.05. Configuración de IPs Virtuales – CARP.....	25
5.06. Sincronización entre Routers – pfSync y HA.....	26
5.07. Configuración de NAT y Reglas de Firewall – VLAN 10.....	27
6. Configuración de los Servicios.....	29
6.01. Configuración del Servidor DHCP - VLAN 10.....	29
6.02. Implementación de los Servidores Web – Apache + PHP.....	30
6.03. Generación del Certificado SSL Autofirmado.....	31
6.04. Configuración de HAProxy – Balanceo de Carga Web.....	32
7. Pruebas realizadas.....	35
7.01. Prueba de Redundancia de Enlace – LAGG.....	35
7.02. Prueba de Redundancia de Servicio Web – HAProxy.....	37
7.03. Prueba de Redundancia de Router – CARP.....	38
7.04. Estado Final de la Red tras las Pruebas.....	39
8. Bibliografía y Referencias.....	41





Leyenda de Colores: Comandos de Consola

¿Qué es?	Color	
Comando		Ruta / Carpeta / Archivo 
Opciones		Red / IP 
Texto / Número		Máscara de Red 
Comentario		VLAN 

LAS REDES OCULTAS QUE HACEN FUNCIONAR LA NUBE

La nube, en realidad, es una red de redes que nunca duerme. Dentro de la nube existen complejas redes virtuales que conectan miles de servidores entre sí, garantizando velocidad, seguridad y disponibilidad. Estas redes permiten que los datos viajen entre distintos centros de datos en cuestión de milisegundos. Gracias a esta infraestructura invisible, los servicios en línea funcionan sin interrupciones, incluso cuando un servidor falla.



1. Introducción a GodCloud

1.01. ¿Cómo empezó todo?

La alta disponibilidad es uno de los pilares fundamentales de cualquier infraestructura tecnológica moderna. Los servicios en la nube que utilizamos a diario (desde plataformas de streaming hasta aplicaciones empresariales críticas) dependen de arquitecturas de red diseñadas para no caer nunca, capaces de tolerar fallos físicos sin que el usuario final perciba ninguna interrupción.

Esta realidad despertó en mí una curiosidad genuina por entender cómo funciona por dentro esa infraestructura invisible. Me pregunté qué ocurre realmente cuando cae un switch en un centro de datos, cómo sigue respondiendo el servicio si un router pierde alimentación, o qué mecanismos garantizan que una conexión TCP activa no se corte durante un failover. Esa curiosidad me llevó a imaginar una topología compleja de switches interconectados en múltiples capas, con caminos redundantes cruzados y routers duplicados, y a plantearme un reto: recrear esa estructura en un entorno virtualizado y estudiar su comportamiento real.

Así nació GodCloud. Un proyecto que no parte de un ejercicio teórico, sino de la necesidad de ver con mis propios ojos cómo se comporta una red empresarial bajo condiciones de fallo, qué protocolos la mantienen en pie y qué limitaciones aparecen cuando se traslada esa arquitectura a un entorno de simulación. El resultado es una infraestructura completa con seis switches Cisco, dos routers pfSense redundantes y una pila de protocolos de alta disponibilidad funcionando de forma conjunta, diseñada para demostrar que es posible construir una red que nunca duerme.

1.02. Repositorio y Gestión del Proyecto

Todo el desarrollo del proyecto ha sido gestionado y documentado a través de dos plataformas complementarias.

El código, configuraciones y archivos del proyecto están disponibles en el repositorio público de GitHub, donde se puede consultar el historial completo de cambios y los archivos de configuración utilizados durante la implementación:

GitHub: <https://github.com/franbarea/GodCloud/tree/main>

La planificación y seguimiento de tareas se realizó mediante un tablero Trello, donde se organizaron las fases del proyecto, las tareas pendientes y los problemas encontrados durante el desarrollo:

Trello: <https://trello.com/b/HFCFj2AO/final-project-god-cloud>



2. Objetivos del Proyecto

2.01. Objetivo General

Diseñar, implementar y verificar una infraestructura de red empresarial de alta disponibilidad en un entorno virtualizado, demostrando que es posible eliminar los puntos únicos de fallo en todas las capas de la arquitectura mediante el uso de protocolos estándar de la industria.

2.02. Objetivos Específicos

Diseño y planificación de la red:

- Diseñar una topología jerárquica de tres capas con múltiples caminos redundantes entre todos los dispositivos.
- Segmentar el tráfico mediante VLANs diferenciando el tráfico de servicios, la gestión de dispositivos y la sincronización entre routers.
- Planificar el direccionamiento IP con subnetting ajustado a las necesidades reales de cada segmento.

Implementación de redundancia en capa 2:

- Configurar PVST+ en los switches Cisco C3640 con módulo NM-16ESW para prevenir bucles de capa 2 en una topología con múltiples enlaces cruzados.
- Establecer S5 como Root Bridge para centralizar y optimizar el árbol de expansión.

Implementación de redundancia en capa 3:

- Configurar LAGG Failover en pfSense agrupando los tres enlaces físicos hacia los switches de distribución en una única interfaz lógica tolerante a fallos.
- Implementar CARP para que los dos routers pfSense compartan una IP virtual como puerta de enlace común, garantizando la continuidad del servicio ante la caída de cualquiera de ellos.
- Configurar pfSync para sincronizar en tiempo real la tabla de estados del firewall, las reglas y el servidor DHCP entre ambos routers.



Implementación de redundancia en capa 7:

- Desplegar tres servidores web Alpine Linux con Apache y PHP84 en la capa de acceso, garantizando la persistencia de la configuración mediante directorios persistentes en los contenedores Docker de GNS3.
- Configurar HAProxy sobre pfSense para distribuir el tráfico web entrante entre los tres servidores mediante balanceo Round Robin, con health checks automáticos para detectar y excluir servidores caídos.
- Implementar SSL/TLS mediante certificado autofirmado generado con una CA propia en pfSense, ofreciendo el servicio exclusivamente por HTTPS con redirección automática desde HTTP.

Verificación y análisis:

- Realizar pruebas de estrés desconectando enlaces y apagando dispositivos para verificar el comportamiento real de los mecanismos de redundancia.
- Documentar las limitaciones encontradas en el entorno de simulación y contrastarlas con el comportamiento esperado en hardware físico real.
- Analizar y justificar las decisiones técnicas tomadas a lo largo del proyecto.



3. Diseño de la Estructura de red

3.01. Diseño Lógico de la Red

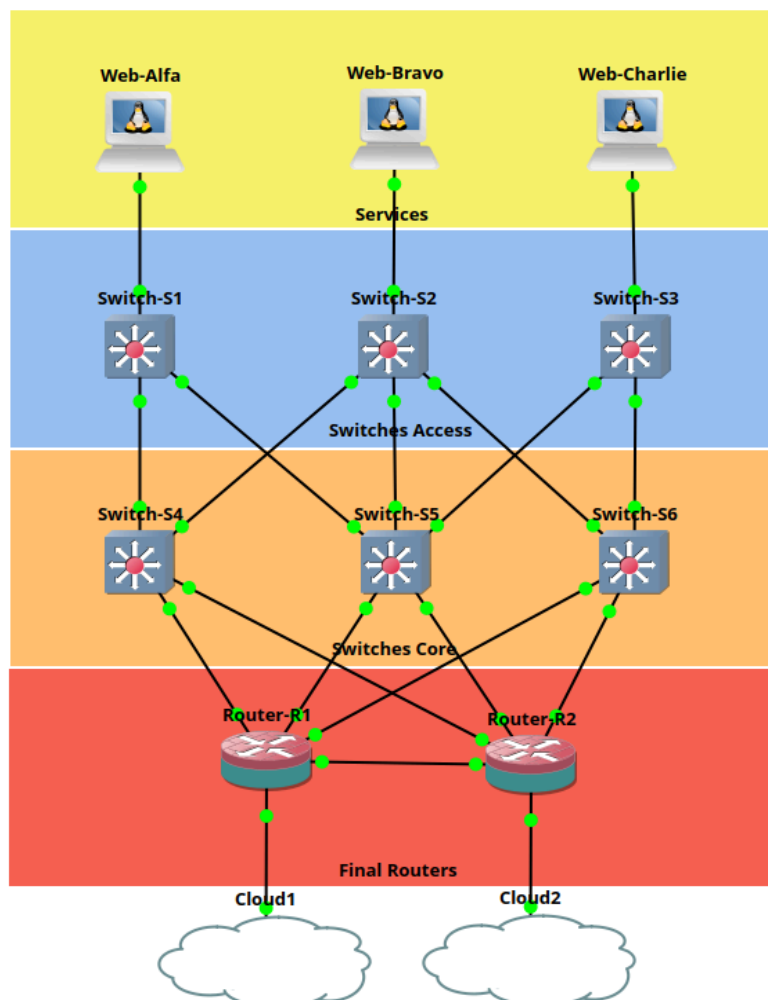
Estructura Conceptual

Servidores Web: Configurados con Alpine Linux como contenedores Docker en GNS3, ejecutando Apache con PHP84 para servir contenido web dinámico. Tres instancias redundantes distribuidas en la capa de acceso sobre VLAN 10, con persistencia de configuración garantizada mediante directorios persistentes del contenedor.

Switches: Configurados con Cisco C3640 y módulo NM-16ESW de 16 puertos con capa L3 y protocolos de Spanning Tree Protocol para prevenir bucles en esta red tan compleja.

Routers: Configurados con pfSense, redundantes mediante protocolos de failover para asegurar conectividad continua y VIP para que mantengan la misma IP compartida de puerta de enlace.

Clouds: Sirven como un adaptador puente en VirtualBox, nos da acceso directo a la red real de nuestro equipo anfitrión, sin restricciones ninguna.



Segmentación de la Red en VLANs

Realizamos una segmentación de la red en VLANs y subnetting con las máscaras para mantener una red más robusta y sencilla de manipular en cuanto a vulnerabilidades.

VLAN 10 (192.168.10.0 /27)

Tráfico del servidor web y clientes.

- **Network:** 192.168.10.0
- **Host Min:** 192.168.10.1
- **Host Max:** 192.168.10.30
- **Broadcast:** 192.168.10.31

VLAN 99 (192.168.99.0 /27)

Administración de los 6 switches (del .11 al .16) y los routers (.2 y .3) con VIP .1.

- **Network:** 192.168.99.0
- **Host Min:** 192.168.99.1
- **Host Max:** 192.168.99.30
- **Broadcast:** 192.168.99.31

RED DE CARP (192.168.100.0 /30)

No es una VLAN, pero es la red del.1 CARP para la sincronización de los routers (.1 y .2)

- **Network:** 192.168.100.0
- **Host Min:** 192.168.100.1
- **Host Max:** 192.168.100.2
- **Broadcast:** 192.168.100.3



Planificación de red para los Switches

Haremos una simulación de **Switches de 16 puertos**, usando máquinas **Cisco C3640** con su módulo **NM-16ESW** de 16 puertos con VLANs. Vamos a ver que conexiones tendrá:

Switch 1 - IP: 192.168.99.11

Dispositivo	Conectado a	Puerto	Modo/Vlan
Switch-S1	Switch-S4	FastEthernet0/0	Troncal (10, 99)
Switch-S1	Switch-S5	FastEthernet0/1	Troncal (10, 99)
Switch-S1	Web Alfa	FastEthernet0/2	Vlan 10
...	...	...	...

Switch 2 - IP: 192.168.99.12

Dispositivo	Conectado a	Puerto	Modo/Vlan
Switch-S2	Switch-S4	FastEthernet0/0	Troncal (10, 99)
Switch-S2	Switch-S5	FastEthernet0/1	Troncal (10, 99)
Switch-S2	Switch-S6	FastEthernet0/2	Troncal (10, 99)
Switch-S2	Web Bravo	FastEthernet0/3	Vlan 10
...	...	...	...

Switch 3 - IP: 192.168.99.13

Dispositivo	Conectado a	Puerto	Modo/Vlan
Switch-S3	Switch-S5	FastEthernet0/0	Troncal (10, 99)
Switch-S3	Switch-S6	FastEthernet0/1	Troncal (10, 99)
Switch-S3	Web Charlie	FastEthernet0/2	Vlan 10
...	...	...	...



Switch 4 - IP: 192.168.99.14

Dispositivo	Conectado a	Puerto	Modo/Vlan
Switch-S4	Switch-S1	FastEthernet0/0	Troncal (10, 99)
Switch-S4	Switch-S2	FastEthernet0/1	Troncal (10, 99)
Switch-S4	Router-R1	FastEthernet0/2	Troncal (10, 99)
Switch-S4	Router-R2	FastEthernet0/3	Troncal (10, 99)
...	...	...	...

Switch 5 - IP: 192.168.99.15

Dispositivo	Conectado a	Puerto	Modo/Vlan
Switch-S5	Switch-S1	FastEthernet0/0	Troncal (10, 99)
Switch-S5	Switch-S2	FastEthernet0/1	Troncal (10, 99)
Switch-S5	Switch-S3	FastEthernet0/2	Troncal (10, 99)
Switch-S5	Router-R1	FastEthernet0/3	Troncal (10, 99)
Switch-S5	Router-R2	FastEthernet0/4	Troncal (10, 99)
...	...	...	...

Switch 6 - IP: 192.168.99.16

Dispositivo	Conectado a	Puerto	Modo/Vlan
Switch-S6	Switch-S2	FastEthernet0/0	Troncal (10, 99)
Switch-S6	Switch-S3	FastEthernet0/1	Troncal (10, 99)
Switch-S6	Router-R1	FastEthernet0/2	Troncal (10, 99)
Switch-S6	Router-R2	FastEthernet0/3	Troncal (10, 99)
...	...	...	...



Planificación de red para los Routers

Haremos una simulación de **Routers de 5 puertos** en Virtualbox, usando máquinas **pfSense**. Vamos a ver que conexiones tendrá:

Router 1 - IP: 192.168.99.2 (VIP: 192.168.99.1)

Dispositivo	Conectado a	Puerto	Vlan
Router-R1	Red Externa	GigaEthernet0/0 (em0)	10, 99
Router-R1	Router-R2	GigaEthernet0/1 (em1)	-
Router-R1	Switch-S4	GigaEthernet0/2 (em2)	10, 99
Router-R1	Switch-S5	GigaEthernet0/3 (em3)	10, 99
Router-R1	Switch-S6	GigaEthernet0/4 (em4)	10, 99

Router 2 - IP: 192.168.99.3 (VIP: 192.168.99.1)

Dispositivo	Conectado a	Puerto	Vlan
Router-R2	Red Externa	GigaEthernet0/0 (em0)	10, 99
Router-R2	Router-R1	GigaEthernet0/1 (em1)	-
Router-R2	Switch-S4	GigaEthernet0/2 (em2)	10, 99
Router-R2	Switch-S5	GigaEthernet0/3 (em3)	10, 99
Router-R2	Switch-S6	GigaEthernet0/4 (em4)	10, 99



3.02. Servicios de redundancia en capa 2

PVST+ (Per-VLAN Spanning Tree Plus) – Switch Cisco C3640

Los switches Cisco C3640 con módulo NM-16ESW utilizan PVST+ para la prevención de bucles de capa 2. Este protocolo mantiene una instancia de árbol de expansión independiente por cada VLAN (10 y 99), eligiendo un switch raíz por VLAN (S5) y bloqueando los puertos redundantes que podrían generar bucles. Ante un fallo de enlace, la convergencia se produce en aproximadamente 30-50 segundos. Cabe destacar que el módulo NM-16ESW no soporta Rapid PVST+ ni MSTP, protocolos que reducirían el tiempo de convergencia a 1-5 segundos. El uso de PVST+ clásico es por tanto una limitación del hardware emulado y no una decisión de diseño. Dado que la redundancia de pasarela está cubierta por CARP en capa 3, los tiempos de convergencia de STP afectan únicamente a fallos internos de la capa de distribución, no a la disponibilidad percibida por los clientes finales.

LAGG (Link Aggregation Group) – Routers pfSense

LAGG en modo failover es un mecanismo de redundancia de enlace que agrupa múltiples interfaces físicas bajo una única interfaz lógica. A diferencia de LACP (IEEE 802.3ad), el modo failover no reparte carga entre los enlaces sino que mantiene un único enlace activo en cada momento, conmutando automáticamente al siguiente disponible cuando el activo falla.

La elección del modo failover frente a LACP no fue arbitraria sino una decisión técnica obligada por las limitaciones del hardware utilizado. El módulo NM-16ESW del Cisco C3640, empleado para simular los switches en GNS3, no soporta la negociación dinámica del protocolo LACP de forma nativa en entorno emulado. Intentar configurar LACP en estos switches resulta en una negociación fallida que deja los puertos sin agregar, haciendo inoperativa la redundancia. El modo failover, al ser estático y no requerir ningún tipo de negociación entre los extremos del enlace, es completamente compatible con este hardware y garantiza la misma redundancia de enlace sin necesidad de configuración adicional en los switches.

En este proyecto, las interfaces em2, em3 y em4 de cada router pfSense se agrupan en una interfaz lagg0, conectadas respectivamente a los switches de distribución S4, S5 y S6. En condiciones normales todo el tráfico circula por em3 hacia S5. Si S5 o su enlace caen, lagg0 conmuta automáticamente a em2 (S4), y si este también falla, a em4 (S6). Este proceso es completamente transparente para las capas superiores, ya que las VLANs y las VIPs CARP están configuradas sobre lagg0 y no sobre las interfaces físicas individuales.



3.03. Servicios de redundancia en capa 3

CARP (Common Address Redundancy Protocol) – Routers pfSense

CARP es un protocolo de redundancia de pasarela que permite que dos o más dispositivos compartan una misma dirección IP virtual (VIP) sobre la red. El protocolo funciona mediante anuncios periódicos entre los miembros del grupo: el dispositivo con menor valor de skew actúa como master y es el único que responde al tráfico destinado a la VIP, mientras el resto permanecen en estado backup monitorizando dichos anuncios. Si el master deja de emitirlos, el backup con el siguiente skew más bajo asume la VIP de forma automática y transparente, sin que los clientes perciban ninguna interrupción.

En este proyecto, R1 actúa como master con skew 0 y R2 como backup con skew 100. Ambos routers comparten dos VIPs: 192.168.10.1 para la VLAN de datos y 192.168.99.1 para la VLAN de gestión, que son las únicas IPs que conocen los clientes y los switches. Las IPs reales de cada router (192.168.10.2 y .3, 192.168.99.2 y .3) se usan exclusivamente para la comunicación interna entre ambos nodos.

VIP (Virtual IP) – Routers pfSense

Las VIPs son las direcciones IP virtuales compartidas por ambos routers mediante CARP. Al estar definidas sobre la interfaz lagg0 en lugar de sobre interfaces físicas, combinan la redundancia de enlace del LAGG con la redundancia de pasarela de CARP, formando una solución de alta disponibilidad en dos niveles: si cae un enlace hacia un switch, lagg0 conmuta de interfaz física; si cae el router completo, CARP transfiere las VIPs al nodo superviviente.

pfSync (Packet Filter State Synchronization) – Routers pfSense

pfSync es el mecanismo que complementa a CARP encargándose de sincronizar en tiempo real la tabla de estados del firewall entre ambos routers a través de la interfaz dedicada em1, en la red 192.168.100.0/30. Gracias a esta sincronización, en el momento en que R2 asume las VIPs tras un fallo de R1, ya dispone de toda la información sobre las conexiones TCP activas, las reglas de firewall y la configuración del servidor DHCP, pudiendo continuar procesándolas sin interrumpirlas.



3.04. Servicios de redundancia en capa 7

HAProxy (High Availability Proxy) – Routers pfSense

HAProxy es un servicio de balanceo de carga y proxy inverso de alta disponibilidad que actúa como punto de entrada único para el tráfico web entrante. Intercepta las peticiones HTTP y HTTPS recibidas en la VIP de CARP y las distribuye entre los servidores web disponibles en la VLAN 10, aplicando algoritmos de balanceo para repartir la carga de forma equitativa. Además de distribuir el tráfico, HAProxy monitoriza continuamente el estado de cada servidor backend mediante health checks periódicos, de modo que si uno de los servidores deja de responder, HAProxy lo marca automáticamente como caído y redirige todo el tráfico al resto de servidores disponibles sin que los clientes perciban ninguna interrupción.

En este proyecto, HAProxy se despliega sobre pfSense y escucha directamente en la VIP compartida por CARP, combinando ambos mecanismos en una solución de alta disponibilidad de dos niveles: si cae un servidor web, HAProxy redirige el tráfico al servidor alternativo; si cae el router completo, CARP transfiere la VIP a R2, que continúa ejecutando HAProxy con la misma configuración gracias a la sincronización de pfSync.



4. Configuración de los Switches Cisco

En el siguiente punto se va a hacer uso de la [Leyenda de Colores](#) que se encuentra en la **página 3**, para poder tener los comandos de configuración bien ordenados visualmente.

Recordar también que estas configuraciones se encuentran subidas en GitHub, dónde las puedes visualizar de mejor manera: [ENLACE](#)

4.01. Configuración de las VLANs

Esta configuración tendremos que hacerla en todos los switches, uno por uno. Nos conectamos a los switches por consola (en un entorno real sería con Putty con cable de consola), y le vamos aplicando los siguientes comandos:

```
! --- Entrar en modo Admin ---
Switch-S*> enable

! --- Entrar en la VLAN database ---
Switch-S*# vlan database

! --- Añadir las vlans en la database---
Switch-S*(vlan) # vlan 10 name SERVICIOS
Switch-S*(vlan) # vlan 99 name GESTION
Switch-S*(vlan) # exit
```

RECORDAR, que después de cada configuración de un switch se tiene que guardar en el archivo de configuración de inicio, para cuando se apague no pierda la configuración. Esto se realiza de la siguiente manera:

```
! --- Guardar Configuración ---
Switch-S1# copy running-config startup-config
```



4.02. Configuración de las interfaces en cada Switch

Configuración del Switch 1

```
Switch-S1# configure terminal
! --- Trunk hacia S4 ---
Switch-S1(config)# interface f0/0
Switch-S1(config-if)# description Trunk_a_S4
Switch-S1(config-if)# switchport mode trunk
Switch-S1(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S1(config-if)# no shutdown

! --- Trunk hacia S5 ---
Switch-S1(config)# interface f0/1
Switch-S1(config-if)# description Trunk_a_S5
Switch-S1(config-if)# switchport mode trunk
Switch-S1(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S1(config-if)# no shutdown

! --- Interfaces de Servicios ---
Switch-S1(config)# interface range f0/2 - 15
Switch-S1(config-if-range)# switchport mode access
Switch-S1(config-if-range)# switchport access vlan 10
Switch-S1(config-if-range)# no shutdown

! --- IP de Gestión ---
Switch-S1(config)# interface vlan 99
Switch-S1(config-if)# ip address 192.168.99.11 255.255.255.224
Switch-S1(config-if)# no shutdown
```



Configuración del Switch 2

```
Switch-S2# configure terminal
! --- Trunk hacia S4 ---
Switch-S2(config)# interface f0/0
Switch-S2(config-if)# description Trunk_a_S4
Switch-S2(config-if)# switchport mode trunk
Switch-S2(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S2(config-if)# no shutdown

! --- Trunk hacia S5 ---
Switch-S2(config)# interface f0/1
Switch-S2(config-if)# description Trunk_a_S5
Switch-S2(config-if)# switchport mode trunk
Switch-S2(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S2(config-if)# no shutdown

! --- Trunk hacia S6 ---
Switch-S2(config)# interface f0/2
Switch-S2(config-if)# description Trunk_a_S6
Switch-S2(config-if)# switchport mode trunk
Switch-S2(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S2(config-if)# no shutdown

! --- Interfaces de Servicios ---
Switch-S2(config)# interface range f0/3 - 15
Switch-S2(config-if-range)# switchport mode access
Switch-S2(config-if-range)# switchport access vlan 10
Switch-S2(config-if-range)# no shutdown

! --- IP de Gestión ---
Switch-S2(config)# interface vlan 99
Switch-S2(config-if)# ip address 192.168.99.12 255.255.255.224
Switch-S2(config-if)# no shutdown
```



Configuración del Switch 3

```
Switch-S3# configure terminal
! --- Trunk hacia S5 ---
Switch-S3(config)# interface f0/0
Switch-S3(config-if)# description Trunk_a_S5
Switch-S3(config-if)# switchport mode trunk
Switch-S3(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S3(config-if)# no shutdown

! --- Trunk hacia S6 ---
Switch-S3(config)# interface f0/1
Switch-S3(config-if)# description Trunk_a_S6
Switch-S3(config-if)# switchport mode trunk
Switch-S3(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S3(config-if)# no shutdown

! --- Interfaces de Servicios ---
Switch-S3(config)# interface range f0/2 - 15
Switch-S3(config-if-range)# switchport mode access
Switch-S3(config-if-range)# switchport access vlan 10
Switch-S3(config-if-range)# no shutdown

! --- IP de Gestión ---
Switch-S3(config)# interface vlan 99
Switch-S3(config-if)# ip address 192.168.99.13 255.255.255.224
Switch-S3(config-if)# no shutdown
```



Configuración del Switch 4

```
Switch-S4# configure terminal
! --- Trunk hacia S1 ---
Switch-S4(config)# interface f0/0
Switch-S4(config-if)# description Trunk_a_S1
Switch-S4(config-if)# switchport mode trunk
Switch-S4(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S4(config-if)# no shutdown

! --- Trunk hacia S2 ---
Switch-S4(config)# interface f0/1
Switch-S4(config-if)# description Trunk_a_S2
Switch-S4(config-if)# switchport mode trunk
Switch-S4(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S4(config-if)# no shutdown

! --- Trunk hacia R1 ---
Switch-S4(config)# interface f0/2
Switch-S4(config-if)# description Trunk_a_R1
Switch-S4(config-if)# switchport mode trunk
Switch-S4(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S4(config-if)# no shutdown

! --- Trunk hacia R2 ---
Switch-S4(config)# interface f0/3
Switch-S4(config-if)# description Trunk_a_R2
Switch-S4(config-if)# switchport mode trunk
Switch-S4(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S4(config-if)# no shutdown

! --- IP de Gestión ---
Switch-S4(config)# interface vlan 99
Switch-S4(config-if)# ip address 192.168.99.14 255.255.255.224
Switch-S4(config-if)# no shutdown
```



Configuración del Switch 5

```
Switch-S5# configure terminal
! --- Trunk hacia S1 ---
Switch-S5(config)# interface f0/0
Switch-S5(config-if)# description Trunk_a_S1
Switch-S5(config-if)# switchport mode trunk
Switch-S5(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S5(config-if)# no shutdown

! --- Trunk hacia S2 ---
Switch-S5(config)# interface f0/1
Switch-S5(config-if)# description Trunk_a_S2
Switch-S5(config-if)# switchport mode trunk
Switch-S5(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S5(config-if)# no shutdown

! --- Trunk hacia S3 ---
Switch-S5(config)# interface f0/2
Switch-S5(config-if)# description Trunk_a_S3
Switch-S5(config-if)# switchport mode trunk
Switch-S5(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S5(config-if)# no shutdown

! --- Trunk hacia R1 ---
Switch-S5(config)# interface f0/3
Switch-S5(config-if)# description Trunk_a_R1
Switch-S5(config-if)# switchport mode trunk
Switch-S5(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S5(config-if)# no shutdown

! --- Trunk hacia R2 ---
Switch-S5(config)# interface f0/4
Switch-S5(config-if)# description Trunk_a_R2
Switch-S5(config-if)# switchport mode trunk
Switch-S5(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S5(config-if)# no shutdown

! --- IP de Gestión y Prioridad en el STP ---
Switch-S5(config)# interface vlan 99
Switch-S5(config-if)# ip address 192.168.99.15 255.255.255.224
Switch-S5(config-if)# no shutdown
Switch-S5(config)# spanning-tree vlan 10 priority 4096
Switch-S5(config)# spanning-tree vlan 99 priority 4096
```



Configuración del Switch 6

```
Switch-S6# configure terminal
! --- Trunk hacia S2 ---
Switch-S6(config)# interface f0/0
Switch-S6(config-if)# description Trunk_a_S2
Switch-S6(config-if)# switchport mode trunk
Switch-S6(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S6(config-if)# no shutdown

! --- Trunk hacia S3 ---
Switch-S6(config)# interface f0/1
Switch-S6(config-if)# description Trunk_a_S3
Switch-S6(config-if)# switchport mode trunk
Switch-S6(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S6(config-if)# no shutdown

! --- Trunk hacia R1 ---
Switch-S6(config)# interface f0/2
Switch-S6(config-if)# description Trunk_a_R1
Switch-S6(config-if)# switchport mode trunk
Switch-S6(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S6(config-if)# no shutdown

! --- Trunk hacia R2 ---
Switch-S6(config)# interface f0/3
Switch-S6(config-if)# description Trunk_a_R2
Switch-S6(config-if)# switchport mode trunk
Switch-S6(config-if)# switchport trunk allowed vlan 1,10,99,1002-1005
Switch-S6(config-if)# no shutdown

! --- IP de Gestión ---
Switch-S6(config)# interface vlan 99
Switch-S6(config-if)# ip address 192.168.99.16 255.255.255.224
Switch-S6(config-if)# no shutdown
```



5. Configuración de los Routers Pfsense

Acceso Inicial mediante Consola VNC

Por defecto, pfSense deshabilita el acceso a la interfaz WebGUI desde la WAN por motivos de seguridad. Para poder configurarlo desde nuestra máquina real a través de la red, necesitamos habilitarlo temporalmente desde la consola VNC de GNS3.

Accedemos al router mediante la consola VNC, seleccionamos la opción 8 (Shell) y ejecutamos los siguientes comandos:

```
RouterR1# pfSsh.php playback enableallowallwan
RouterR1# easyrule pass wan tcp any any 443
RouterR1# easyrule pass wan tcp any any 80
```

Una vez terminada la configuración completa, estas reglas deben eliminarse desde **Firewall** → **Rules** → **WAN** para restaurar la seguridad perimetral.

Configuración Inicial mediante WebGUI

Desde la consola VNC podemos ver la IP asignada a la interfaz WAN por el DHCP de nuestra red real. Accedemos al WebGUI escribiendo esa IP en cualquier navegador:

https://IP_WAN_ASIGNADA

Al entrar por primera vez, pfSense muestra un asistente de configuración inicial donde configuramos:

- Hostname: Router-R1 (o Router-R2 en el segundo router)
- Timezone: Europe/Madrid
- DNS: 8.8.8.8
- Contraseña de administrador: pfsensegodcloud



5.01. Interfaz de Sincronización CARP – em1

La interfaz em1 se usa exclusivamente para la sincronización directa entre R1 y R2 mediante pfSync y CARP. Al ser un enlace punto a punto usamos una red /30 que solo necesita dos IPs.

Nos vamos a **Interfaces** → **Assignments** → **Add**, seleccionamos em1 y guardamos. Luego entramos en la interfaz recién creada y configuramos:

- Enable: ☒ Marcado
- Description: SYNC
- IPv4 Configuration: Static
 - R1: 192.168.100.1 / 30
 - R2: 192.168.100.2 / 30
- Gateway: none

5.02. Agrupación de Interfaces – LAGG Failover

Antes de crear el LAGG, verificamos que em2, em3 y em4 no están asignadas como interfaces en **Interfaces** → **Assignments**. Si lo están, las eliminamos.

Nos vamos a **Interfaces** → **LAGGs** → **Add** y configuramos:

- Members: em2, em3, em4
- LAGG Protocol: Failover
- Description: LAGG_SWITCHES



5.03. Segmentación Virtual – VLANs sobre LAGG

Creemos las dos VLANs sobre la interfaz lagg0 en **Interfaces** → **VLANs** → **Add**:

VLAN 10:

- Parent: lagg0
- VLAN Tag: 10
- Description: SERVICIOS

VLAN 99:

- Parent: lagg0
- VLAN Tag: 99
- Description: GESTION

5.04. Asignación y Direccionamiento de Interfaces

Añadimos las dos interfaces en **Interfaces** → **Assignments** → **Add**, aparecerán como OPT1 y OPT2. Las configuramos así:

OPT1 → SERVICIOS (lagg0.10):

- Enable: ☒ Marcado
- Description: SERVICIOS
- IPv4 Configuration: Static
 - R1: 192.168.10.1 / 27
 - R2: 192.168.10.2 / 27
- Gateway: none

OPT2 → GESTION (lagg0.99):

- Enable: ☒ Marcado
- Description: SERVICIOS
- IPv4 Configuration: Static
 - R1: 192.168.99.1 / 27
 - R2: 192.168.99.2 / 27
- Gateway: none



5.05. Configuración de IPs Virtuales – CARP

Las VIPs son las IPs virtuales compartidas por ambos routers que verán los clientes. Las creamos en **Firewall → Virtual IPs → Add**:

VIP de SERVICIOS (VLAN 10):

- Type: CARP
- Interface: SERVICIOS (lagg0.10)
- Address: 192.168.10.1 / 27
- Password: godcloud.1234
- VHID: 1
- Base: 1
- Skew (define la prioridad de cada nodo para ser el principal.)
 - Skew en R1: 0
 - Skew en R2: 100
- Description: VIP_SERVICIOS

VIP de GESTION (VLAN 99):

- Type: CARP
- Interface: GESTION (lagg0.99)
- Address: 192.168.99.1 / 27
- Password: godcloud.1234
- VHID: 2
- Base: 1
- Skew (define la prioridad de cada nodo para ser el principal.)
 - Skew en R1: 0
 - Skew en R2: 100
- Description: VIP_GESTION



5.06. Sincronización entre Routers – pfSync y HA

pfSync sincroniza en tiempo real los estados del firewall, las reglas y el DHCP entre ambos routers. Nos vamos a **System** → **High Availability Sync**:

En ambos routers:

- Synchronize states: ☒ Marcado
- Sync Interface: SYNC (em1)
- Peer IP (Apuntando a la dirección del otro)
 - R1: 192.168.100.2
 - R2: 192.168.100.1

Solo en R1 (MASTER)

- Synchronize config to IP: 192.168.100.2
- Username: admin
- Password: pfsensegodcloud
- Synchronize admin: ☒ Marcado
- Firewall rules: ☒ Marcado
- DHCP Server settings: ☒ Marcado
- Virtual IPs: ☒ Marcado
- Certificate Authorities, Certificates, and Certificate Revocation Lists: ☒ Marcado
- NAT configuration: ☒ Marcado

PROBLEMA: La configuración de HAProxy no se sincroniza mediante pfSync al ser un paquete externo. Es necesario configurar manualmente el backend y los frontends en R2 con los mismos valores que R1.



5.07. Configuración de NAT y Reglas de Firewall – VLAN 10

Política de Seguridad Aplicada

Se aplica una política de mínimo privilegio sobre el tráfico de la VLAN 10. El tráfico de salida se permite de forma general para que los servidores puedan resolver DNS, actualizarse y comunicarse con el exterior. El tráfico de entrada desde la WAN se restringe únicamente a los puertos estrictamente necesarios para el servicio web, bloqueando cualquier otro acceso no autorizado.

NAT Outbound – Salida a Internet

Al cambiar el modo a **Manual Outbound NAT** en **Firewall → NAT → Outbound**, pfSense genera automáticamente las reglas de salida para todas las interfaces configuradas. Se revisaron las reglas generadas automáticamente y se eliminaron las correspondientes a la red SYNC, ya que esta red es un enlace punto a punto privado exclusivamente para la sincronización entre R1 y R2 y no debe tener salida a internet por motivos de seguridad.

Las reglas de NAT Outbound que permanecen activas son:

- Synchronize config to IP: 192.168.100.2
- Username: admin
- Password: pfsensegodcloud
- Synchronize admin: ☒ Marcado
- Firewall rules: ☒ Marcado
- DHCP Server settings: ☒ Marcado
- Virtual IPs: ☒ Marcado



Reglas de Firewall – Tráfico de Entrada (WAN)

El tráfico entrante desde internet se restringe exclusivamente a los puertos del servicio web. Nos vamos a **Firewall** → **Rules** → **WAN** → **Add** y creamos las siguientes reglas en orden:

- **Reglas 1 y 2:** Permiten todo el tráfico de entrada al servicio web (HTTP y HTTPS)
- **Regla 3 y 4:** Estas reglas **NO SON SEGURAS** pero para las pruebas las necesito para poder acceder a la configuración del router, y ver el estado de este.
- **Regla 5:** Por seguridad, bloquea el resto del tráfico

Rule	Action	Interface	Protocol	Source	Destination	Port
REGLA 1 (Permitir)	Pass	WAN	TCP	Any	192.168.10.1	80
REGLA 2 (Permitir)	Pass	WAN	TCP	Any	192.168.10.1	443
REGLA 3 (Permitir)	Pass	WAN	TCP	Any	WAN Addresses	80
REGLA 4 (Permitir)	Pass	WAN	TCP	Any	WAN Addresses	443
REGLA 5 (Bloquear)	Block	WAN	Any	Any	Any	Any

Reglas de Firewall – Tráfico de Salida (VLAN 10)

El tráfico saliente hacia internet se permitirá para que los servidores webs puedan descargar paquetes o actualizar el sistema operativo. Nos vamos a **Firewall** → **Rules** → **SERVICIOS** → **Add** y creamos las siguientes reglas en orden:

- **Regla 1:** Permite todo el tráfico de salida de los servidores webs

Rule	Action	Interface	Protocol	Source	Destination	Port
REGLA 1 (Permitir)	Pass	SERVICIOS	Any	SERVICIOS net	Any	Any



6. Configuración de los Servicios

6.01. Configuración del Servidor DHCP - VLAN 10

Pool dinámico general

Este pool es solamente por si acaso o para configuraciones previas, ya que para la mayoría de servidores de servicios lo mejor es que sean fijos según su MAC.

Nos vamos a **Services** → **DHCP Server** → **SERVICIOS** y configuramos el pool general para los equipos clientes, usando de puerta de enlace y DNS a nuestra VIP de CARP:

- Enable: ☒ Marcado
- Subnet: 192.168.10.0 / 27
- Range From: 192.168.10.20
- Range To: 192.168.10.30
- Gateway: 192.168.10.1
- DNS Primario: 1.1.1.1
- DNS Secundario: 192.168.10.1

Reservas estáticas para los servidores web

Las reservas garantizan que cada servidor web reciba siempre la misma IP basándose en su dirección MAC. Esto es imprescindible para que HAProxy pueda apuntar a IPs fijas y estables.

Dentro de la misma página del DHCP bajamos hasta la sección **DHCP Static Mappings** y añadimos una entrada por cada servidor con **Add**:

MAC Address	IP Address	Hostname	Description
02:42:77:2b:a7:00	192.168.10.10	Web-Alfa	Servidor Web Alfa
02:42:42:70:a5:00	192.168.10.11	Web-Bravo	Servidor Web Bravo
02:42:69:63:3e:00	192.168.10.12	Web-Charlie	Servidor Web Charlie



6.02. Implementación de los Servidores Web – Apache + PHP

Estructura de los servidores

Se despliegan tres máquinas virtuales Alpine Linux en GNS3 como contenedores Docker conectadas a la capa de acceso en VLAN 10., así estarán conectadas:

Dispositivo	Conectado a	Puerto	Modo/Vlan
Switch-S1	Web Alfa	FastEthernet0/2	Vlan 10
Switch-S2	Web Bravo	FastEthernet0/3	Vlan 10
Switch-S3	Web Charlie	FastEthernet0/2	Vlan 10

Persistencia de la configuración

Los contenedores Docker en GNS3 no persisten cambios entre reinicios por diseño. Para solucionar esto, configuramos dos directorios persistentes en cada contenedor desde **Edit** → **Configure** → **Advanced**:

```
/var/www/localhost/htdocs  
/etc/apache2
```

Start Command

En el campo **Start command** de cada contenedor configuramos el siguiente comando, que instala Apache y PHP automáticamente en cada arranque y deja la consola disponible:

```
/bin/sh -c "apk add --no-cache apache2 php84 php84-apache2 && httpd -D  
FOREGROUND & /bin/sh"
```

Página Web con PHP

La página detecta automáticamente el hostname del servidor y muestra su IP y última actualización. Creamos el archivo index.php en cada servidor:

```
Web-*# vi /var/www/localhost/htdocs/index.php
```

El archivo index.php está disponible en el repositorio GitHub del proyecto: [ENLACE](#)



Habilitar PHP en Apache

Editamos la configuración de Apache para que procese archivos PHP y los sirva por defecto:

```
Web-*# vi /etc/apache2/httpd.conf
LoadModule php_module modules/mod_php84.so
AddType application/x-httpd-php .php
...
DirectoryIndex index.php index.html
```

6.03. Generación del Certificado SSL Autofirmado

HAProxy necesita un certificado SSL para poder ofrecer el servicio por HTTPS en el puerto 443. Al ser un entorno de laboratorio generamos un certificado autofirmado desde el propio pfSense. La diferencia con un certificado de una CA pública como Let's Encrypt es que el navegador mostrará un aviso de seguridad la primera vez, pero el cifrado es exactamente igual de robusto.

Crear la Autoridad Certificadora (CA)

Antes de crear el certificado del servidor necesitamos una CA propia que lo firme. Nos vamos a **System** → **Cert Manager** → **CAs** → **Add**:

- Descriptive Name: GodCloud
- Method: Create an internal Certificate Authority
- Key Type: RSA
- Key Length: 2048
- Digest Algorithm: SHA256
- Lifetime: 3650 días (10 años)
- Common Name: GodCloud-CA
- Y los demás datos de la "organización"



Crear el Certificado del Servidor

Nos vamos a **System** → **Cert Manager** → **Certificates** → **Add**:

- Method: Create an internal Certificate
- Descriptive Name: GodCloud-CERT
- Certificate Authority: GodCloud
- Key Type: RSA
- Key Length: 2048
- Digest Algorithm: SHA256
- Lifetime: 398 días
- Common Name: godcloud.net
- Y los demás datos de la "organización"

Bajamos hasta **Certificate Attributes**:

- Certificate Type: Server Certificate
- Alternative Names: IP Addresses: 192.168.10.1

6.04. Configuración de HAProxy – Balanceo de Carga Web

1. Instalación de HAProxy

Nos vamos a **System** → **Package Manager** → **Available Packages**, buscamos haproxy e instalamos el paquete haproxy.

→ **Install** → **Confirm**

La configuración de HAProxy no se sincroniza mediante pfSync al ser un paquete externo. Es necesario configurar manualmente el backend y los frontends en R2 con los mismos valores que R1, sin embargo, los certificados si que los sincroniza.

2. Configuración del Backend

El backend define los servidores web reales entre los que HAProxy repartirá el tráfico. Nos vamos a **Services** → **HAProxy** → **Backend** → **Add**:

Configuración general:

- Name: godcloud_backend
- Balance: Round Robin



Servidores del backend:

Añadimos los tres servidores en la sección Server list con **Add**:

Hostname	IP Address	Port	State
Web-Alfa	192.168.10.10	80	Active
Web-Bravo	192.168.10.11	80	Active
Web-Charlie	192.168.10.12	80	Active

Health Check:

- Health check method: HTTP
- Check frequency: 2000 ms
- Http check method: GET
- Log checks: ☒ Marcado

3. Configuración del Frontend HTTP

El frontend HTTP escucha en el puerto 80 y redirige automáticamente todo el tráfico a HTTPS. Nos vamos a **Services** → **HAProxy** → **Frontend** → **Add**:

- Name: godcloud_http
- Description: Frontend HTTP – Redirección a HTTPS
- Status: Active
- External address: 192.168.10.1 puerto 80
- Type: HTTP / HTTPS

Acciones – Redirección HTTP a HTTPS:

En la sección **Actions** añadimos:

- Action: http-request redirect
- Rule: scheme https code 301



4. Configuración del Frontend HTTPS

El frontend HTTPS escucha en el puerto 443, termina el SSL y reenvía el tráfico en claro al backend. Nos vamos a **Services** → **HAProxy** → **Frontend** → **Add**:

- Name: godcloud_https
- Description: Frontend HTTPS – Balanceo SSL
- Status: Active
- External address: 192.168.10.1 puerto 443
- SSL Offloading: ☒ Marcado
- Type: HTTP / HTTPS

Backend por defecto:

En la sección debajo de **Actions** añadimos:

- Default Backend: godcloud_backend

Configuración SSL:

En la sección **Actions** añadimos:

- Certificate: GodCloud-CERT
- Add ACL for certificate Subject Alternative Names: ☒ Marcado

5. Configuración general de HAProxy

Nos vamos a **Services** → **HAProxy** → **Settings**:

- Enable HAProxy: ☒ Marcado
- Maximum connections: 1000
- Internal stats port: 2121



7. Pruebas realizadas

7.01. Prueba de Redundancia de Enlace – LAGG

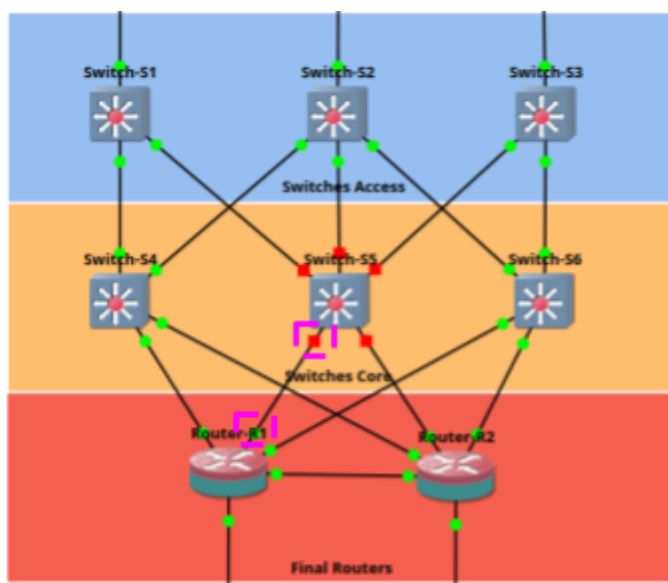
Funcionamiento del LAGG

El mecanismo LAGG en modo Failover agrupa las interfaces físicas em2, em3 y em4 bajo una única interfaz lógica lagg0. En condiciones normales, pfSense designa una interfaz como master y la mantiene como enlace activo, usando las demás como respaldo inactivo. Cuando la interfaz activa pierde su enlace físico, el sistema conmuta automáticamente a la siguiente interfaz disponible de forma transparente para las capas superiores, manteniendo las VLANs y las VIPs CARP operativas sin interrupción.

Limitación en el Entorno GNS3/QEMU

Durante las pruebas se detectó que al apagar un nodo switch directamente desde GNS3, el hipervisor QEMU no propaga la señal eléctrica de link-down al sistema operativo pfSense. Como consecuencia, pfSense sigue considerando la interfaz como activa aunque el switch de destino esté completamente apagado, impidiendo que el LAGG conmute automáticamente. Esta limitación es exclusiva del entorno de virtualización y no afectaría a un despliegue sobre hardware físico real, donde la caída de tensión en el puerto generaría una interrupción de hardware inmediata.

Para ver la configuración del LAGG entramos a **Diagnostics** → **Command Prompt** ejecutamos el siguiente comando: `ifconfig lagg0`



Shell Output - ifconfig lagg0

```
lagg0: flags=1008943<UP, BROADCAST, RUNNING, PROMISC, ...
options=48120bb<RXCSUM, TXCSUM, VLAN_MTU, VLAN_...
ether 0c:0f:e2:24:00:03
hwaddr 00:00:00:00:00:00
inet6 fe80::eab:99ff:fe8a:3%lagg0 prefixlen 64
laggproto failover lagghash l2,l3,l4
laggport: em2 flags=0<>
laggport: em3 flags=5<MASTER, ACTIVE>
laggport: em4 flags=0<>
groups: lagg
media: Ethernet autoselect
status: active
nd6 options=21<PERFORMNUD, AUTO_LINKLOCAL>
```

A pesar de estar la máquina apagada, GNS3 aún marca el nodo del router como activo



Solución Aplicada para la Demostración

Para poder demostrar el funcionamiento real del LAGG Failover dentro de las limitaciones de GNS3, se optó por suspender el cable virtual entre el switch activo y el router en lugar de apagar el nodo. Esta acción sí genera correctamente la señal de link-down a nivel de interfaz, permitiendo que pfSense la detecte y conmute automáticamente al siguiente enlace disponible.

El resultado observado fue el siguiente:

Antes de suspender el cable:

Shell Output - ifconfig lagg0

```
lagg0: flags=1008943<UP, BROADCAST, RUNNING, PROMISC, ...>
options=48120bb<RXCSUM, TXCSUM, VLAN_MTU, VLAN_HWTSTAMP, ...>
ether 0c:0f:e2:24:00:03
hwaddr 00:00:00:00:00:00
inet6 fe80::eab:99ff:fe8a:3%lagg0 prefixlen 64
laggproto failover lagghash l2, l3, l4
laggport: em2 flags=0<>
laggport: em3 flags=5<MASTER, ACTIVE>
laggport: em4 flags=0<>
groups: lagg
media: Ethernet autoselect
status: active
nd6 options=21<PERFORMNUD, AUTO_LINKLOCAL>
```

Tras suspender el cable em3 → S5:

Shell Output - ifconfig lagg0

```
lagg0: flags=1008943<UP, BROADCAST, RUNNING, PROMISC, ...>
options=48120bb<RXCSUM, TXCSUM, VLAN_MTU, VLAN_HWTSTAMP, ...>
ether 0c:0f:e2:24:00:03
hwaddr 00:00:00:00:00:00
inet6 fe80::e0f:e2ff:fe24:3%lagg0 prefixlen 64
laggproto failover lagghash l2, l3, l4
laggport: em2 flags=0<>
laggport: em3 flags=1<MASTER>
laggport: em4 flags=4<ACTIVE>
groups: lagg
media: Ethernet autoselect
status: active
nd6 options=21<PERFORMNUD, AUTO_LINKLOCAL>
```



7.02. Prueba de Redundancia de Servicio Web – HAProxy

Funcionamiento del HAProxy

HAProxy monitoriza continuamente el estado de cada servidor backend mediante health checks periódicos. En condiciones normales, el tráfico web se distribuye entre los tres servidores mediante Round Robin. Cuando uno de los servidores deja de responder, HAProxy lo marca automáticamente como caído y redirige todo el tráfico a los servidores restantes sin que los clientes perciban ninguna interrupción.

Procedimiento de la Prueba

Para demostrar el funcionamiento, se apagó el servidor Web-Bravo y el Switch-S3 que da acceso al servidor Web-Charlie, apagados directamente desde GNS3 mientras el servicio estaba en funcionamiento y accesible desde la web <https://godcloud.net>, se verificó el comportamiento de HAProxy en **Services** → **HAProxy** → **Stats**.

Antes de apagar Web-Bravo:

godcloud_backend_ipvANY																						
		Queue			Session rate			Sessions					Bytes		Denied		Errors			Warnings		Status
		Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	
☐	Web-Alfa	0	0	-	0	7	0	1	-	62	62	54m34s	41 069	248 740		0		0	0	0	0	5m40s UP
☐	Web-Bravo	0	0	-	0	2	0	1	-	11	11	54m28s	7 368	44 792		0		0	0	0	0	5m40s UP
☐	Web-Charlie	0	0	-	0	2	0	1	-	11	11	54m39s	7 368	44 836		0		0	0	0	0	5m40s UP
	Backend	0	0		0	7	0	1	100	90	84	54m28s	59 479	339 670	0	0		6	0	0	0	5m40s UP

Tras apagar Web-Bravo:

godcloud_backend_ipvANY																							
☐		Queue			Session rate			Sessions						Bytes		Denied		Errors			Warnings		Status
		Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	
☐	Web-Alfa	0	0	-	0	7		0	1	-	62	62	59m28s	41 069	248 740		0		0	0	0	0	10m34s UP
☐	Web-Bravo	0	0	-	0	2		0	1	-	11	11	59m22s	7 368	44 792		0		0	0	0	0	30s DOWN
☐	Web-Charlie	0	0	-	0	2		0	1	-	11	11	59m33s	7 368	44 836		0		0	0	0	0	3s UP
	Backend	0	0		0	7		0	1	100	90	84	59m22s	59 479	339 670	0	0		6	0	0	0	10m34s UP

Tras apagar el Switch-S3 que da acceso a Web-Charlie:

godcloud_backend_ipvANY																						
☐		Queue			Session rate			Sessions					Bytes		Denied		Errors			Warnings		Status
		Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	
☐	Web-Alfa	0	0	-	0	7	0	1	-	62	62	1h1m	41 069	248 740		0		0	0	0	0	12m28s UP
☐	Web-Bravo	0	0	-	0	2	0	1	-	11	11	1h1m	7 368	44 792		0		0	0	0	0	2m24s DOWN
☐	Web-Charlie	0	0	-	0	2	0	1	-	11	11	1h1m	7 368	44 836		0		0	0	0	0	9s DOWN
	Backend	0	0		0	7	0	1	100	90	84	1h1m	59 479	339 670	0	0		6	0	0	0	12m28s UP



7.03. Prueba de Redundancia de Router – CARP

Funcionamiento del CARP

CARP mantiene un anuncio periódico entre R1 y R2 para determinar qué nodo actúa como MASTER. R1, con skew 0, es el nodo principal y el único que responde al tráfico destinado a las VIPs. R2, con skew 100, permanece en estado BACKUP monitorizando los anuncios de R1. Si R1 deja de emitirlos, R2 asume las VIPs automáticamente y continúa procesando el tráfico gracias a la sincronización previa de pfSync.

Procedimiento de la Prueba

Se verificó el estado CARP inicial en **Status** → **CARP** y posteriormente se apagó R1 desde GNS3, observando el comportamiento de R2.

Antes de apagar R1 – Estado en Router-R1:

CARP Status			
Interface and VHID	Virtual IP Address	Description	Status
SERVICIOS@1	192.168.10.1/27	VIP_SERVICIOS	MASTER
GESTION@2	192.168.99.1/27	VIP_GESTION	MASTER
State Synchronization Status			
State Creator Host IDs:			
• 7d1e8631 (This node)			
• be79a74c			

Antes de apagar R1 – Estado en Router-R2:

CARP Status			
Interface and VHID	Virtual IP Address	Description	Status
SERVICIOS@1	192.168.10.1/27	VIP_SERVICIOS	BACKUP
GESTION@2	192.168.99.1/27	VIP_GESTION	BACKUP
State Synchronization Status			
State Creator Host IDs:			
• 7d1e8631			
• be79a74c (This node)			

Tras apagar R2 – Estado en Router-R2:

CARP Status			
Interface and VHID	Virtual IP Address	Description	Status
SERVICIOS@1	192.168.10.1/27	VIP_SERVICIOS	MASTER
GESTION@2	192.168.99.1/27	VIP_GESTION	MASTER
State Synchronization Status			
State Creator Host IDs:			
• 7d1e8631			
• be79a74c (This node)			



7.04. Estado Final de la Red tras las Pruebas

Estado de la red

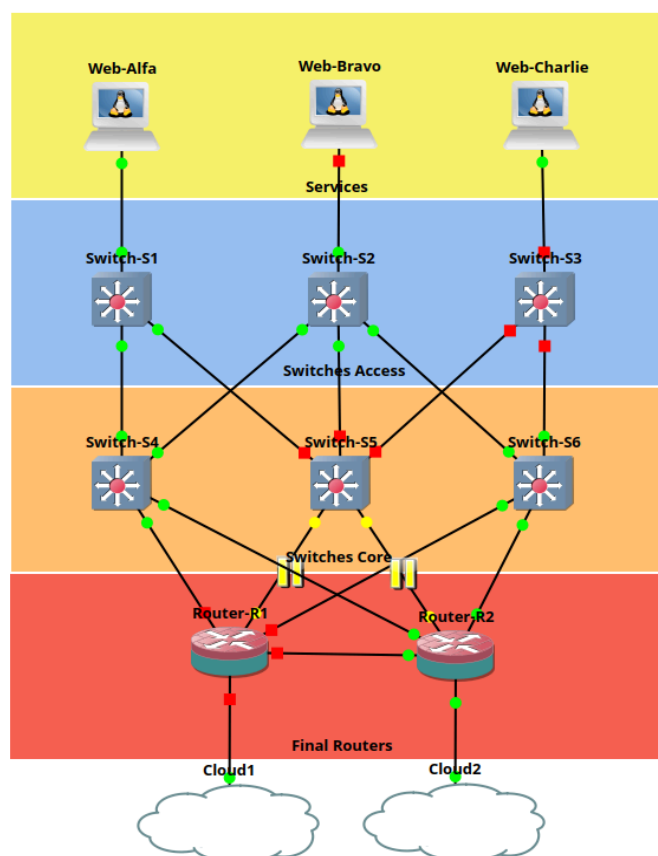
A lo largo de las pruebas realizadas, la infraestructura GodCloud ha demostrado un comportamiento consistente y progresivamente más robusto en cada escenario evaluado.

En la **primera prueba** se verificó el LAGG Failover suspendiendo el cable virtual entre S5 y R1, comprobando que pfSense conmutó automáticamente al siguiente enlace disponible sin interrumpir las capas superiores.

En la **segunda prueba** se apagó Web-Bravo y adicionalmente Switch-S3, verificando que HAProxy detectó la caída del servidor y redirigió el tráfico a los servidores disponibles, demostrando simultáneamente la redundancia a nivel de servidor web y a nivel de switch de acceso.

En la **tercera prueba** se apagó R1 y se verificó mediante la web y el estado de CARP que asumió correctamente las VIPs CARP y continuó sirviendo el tráfico web sin interrupción.

En ninguna de las tres pruebas el servicio dejó de estar disponible, lo que confirma que los mecanismos de alta disponibilidad implementados en las tres capas funcionan de forma conjunta y coordinada tal como fue diseñado.



Conclusión / Reflexión final

El desarrollo de GodCloud en un entorno de simulación cerrado ha permitido implementar y verificar una infraestructura de alta disponibilidad completa, pero también ha puesto de manifiesto las limitaciones inherentes a este tipo de entornos. La ausencia de una IP pública real y de un servidor DNS externo impide exponer el servicio fuera de la red local, haciendo necesario recurrir a resolución manual mediante el archivo `/etc/hosts` del cliente. Estas limitaciones son propias del entorno de laboratorio y no afectarían a un despliegue real sobre infraestructura física con conectividad a internet.

Otra limitación relevante es la del hardware emulado. Los switches Cisco C3640 con módulo NM-16ESW no soportan protocolos modernos como LACP (IEEE 802.3ad), lo que obligó a utilizar LAGG en modo failover en lugar de agregación dinámica de enlaces. En un entorno de producción real, con switches de gama actual, sería posible implementar LACP y obtener además distribución de carga entre los enlaces, mejorando el rendimiento además de la redundancia.

Por otro lado, el uso de seis switches organizados en dos capas jerárquicas ha demostrado ser fundamental para garantizar la redundancia física de la red. Cada servidor web tiene conectividad a través de un switch de acceso independiente, y cada switch de acceso está conectado a múltiples switches de distribución, eliminando cualquier punto único de fallo en la capa de switching. Esta arquitectura, habitual en entornos empresariales reales, sería inviable con un número reducido de dispositivos y justifica la complejidad topológica del proyecto.

GodCloud representa apenas una fracción de lo que una infraestructura de red empresarial puede llegar a ser. El proyecto podría extenderse indefinidamente: añadir un túnel VPN sobre la VLAN 99 para cifrar el tráfico de gestión entre administradores remotos y los dispositivos de red, incorporar servicios adicionales como balanceo de bases de datos, servidores de correo o sistemas de monitorización con sus propios mecanismos de redundancia, o simplemente escalar la topología agregando más switches, más routers y más servidores web hasta construir una red de la complejidad que se desee. La arquitectura jerárquica diseñada en GodCloud está preparada para crecer sin límite, añadiendo capas de acceso, distribución y núcleo tantas veces como sea necesario. Es precisamente esa capacidad de expansión infinita, junto con el control absoluto sobre cada capa de la infraestructura, lo que convierte a este tipo de proyectos en algo cercano al poder del dios en la nube: una red que nunca duerme, que nunca falla, y que puede crecer tanto como la imaginación lo permita.



8. Bibliografía y Referencias

Documentación de mi proyecto realizado en otras plataformas:

[1] GitHub. Disponible en: <https://github.com/franbarea/GodCloud/tree/main>

[2] Trello. Disponible en: <https://trello.com/b/HFCFj2AO/final-project-god-cloud>

Documentación oficial técnica:

[1] Netgate Documentation. "CARP - Common Address Redundancy Protocol". Disponible en: <https://docs.netgate.com/pfsense/en/latest/highavailability/carp.html>

[2] Netgate Documentation. "Configuring pfSync". Disponible en: <https://docs.netgate.com/pfsense/en/latest/highavailability/pfsync.html>

[3] Netgate Documentation. "LAGG Interfaces". Disponible en: <https://docs.netgate.com/pfsense/en/latest/interfaces/laqg.html>

[4] GNS3 Documentation. "QEMU VM Setup in GNS3". Disponible en: <https://docs.gns3.com/docs/emulators/qemu>

[5] GNS3 Documentation. "GNS3 Setup Wizard". Disponible en: <https://docs.gns3.com>

[6] Cisco Systems. "Cisco IOS Switching Services Configuration Guide – Spanning Tree Protocol". Disponible en: https://www.cisco.com/c/en/us/td/docs/ios/12_2/switch/configuration/guide/fswtch_c.html

[7] Netgate Documentation. "HAProxy Package". Disponible en: <https://docs.netgate.com/pfsense/en/latest/packages/haproxy.html>

[8] Alpine Linux Documentation. "Alpine Linux Docker Image". Disponible en: <https://wiki.alpinelinux.org/wiki/Docker>

[9] The Apache Software Foundation. "Apache HTTP Server Documentation". Disponible en: <https://httpd.apache.org/docs/>



Herramientas y Software Utilizados

- [10] GNS3. Graphical Network Simulator 3. Versión 2.x. Disponible en: <https://www.gns3.com>
- [11] Netgate. pfSense Community Edition. Versión 2.7.x. Disponible en: <https://www.pfsense.org>
- [12] Wireshark Foundation. Wireshark Network Analyzer. Disponible en: <https://www.wireshark.org>
- [13] HAProxy Technologies. HAProxy Load Balancer. Versión 2.8.x. Disponible en: <https://www.haproxy.org>
- [14] Alpine Linux. Alpine Linux OS. Versión 3.x. Disponible en: <https://www.alpinelinux.org>

Estándares y RFCs

- [15] IEEE. IEEE 802.1D – Media Access Control (MAC) Bridges and Virtual Bridged Local Area Networks – Spanning Tree Protocol. Institute of Electrical and Electronics Engineers, 2004.
- [16] IEEE. IEEE 802.3ad – Link Aggregation. Institute of Electrical and Electronics Engineers, 2000.
- [17] IETF. RFC 3768 – Virtual Router Redundancy Protocol (VRRP). Protocolo de referencia en el que se basa CARP. Disponible en: <https://www.rfc-editor.org/rfc/rfc3768>
- [18] IEEE. IEEE 802.1Q – Virtual LANs (VLANs). Institute of Electrical and Electronics Engineers, 2022.

Documentación Complementaria

- [19] Cisco Systems. "Understanding and Configuring VLANs". Cisco Documentation. Disponible en: <https://www.cisco.com/c/en/us/support/docs/lan-switching/vlan/10023-3.html>
- [20] Netgate Forum. "pfSense High Availability Setup". Disponible en: <https://forum.netgate.com>

